

SECR2043 OPERATING SYSTEMS

Name : NG YU HIN
Student ID : A23CS0148
Section : 02

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or Mac OS) environment:

```
wget -c https://t.ly/mainprocess.c
wget -c https://t.ly/subprocess1.c
wget -c https://t.ly/subprocess2.c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press enter two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

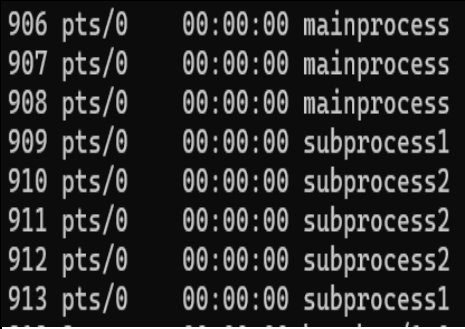
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

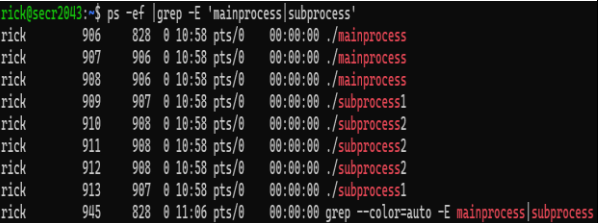
Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command	Output
<code>ps e</code>	

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command	Output
<code>ps -ef grep -E 'mainprocess subprocess'</code>	

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command	Output
<code>ps -ef grep -E 'subprocess'</code>	<pre>rick@secre2043:~\$ ps -ef grep -E 'subprocess' rick 909 907 0 10:58 pts/0 00:00:00 ./subprocess1 rick 910 908 0 10:58 pts/0 00:00:00 ./subprocess2 rick 911 908 0 10:58 pts/0 00:00:00 ./subprocess2 rick 912 908 0 10:58 pts/0 00:00:00 ./subprocess2 rick 913 907 0 10:58 pts/0 00:00:00 ./subprocess1 rick 947 828 0 11:09 pts/0 00:00:00 grep --color=au to -E subprocess</pre>

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command	Output
<code>ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess'</code>	<pre>rick@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd grep -E 'subprocess' 909 rick 0.0 0.1 ./subprocess1 910 rick 0.0 0.1 ./subprocess2 911 rick 0.0 0.1 ./subprocess2 912 rick 0.0 0.1 ./subprocess2 913 rick 0.0 0.1 ./subprocess1 955 rick 0.0 0.1 grep --color=auto -E subprocess</pre>

Question 5

Building on the `ps` command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command	Output
<code>ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess'</code>	<pre>rick@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep -E 'subprocess' 909 rick 0.0 0.1 ./subprocess1 910 rick 0.0 0.1 ./subprocess2 911 rick 0.0 0.1 ./subprocess2 912 rick 0.0 0.1 ./subprocess2 913 rick 0.0 0.1 ./subprocess1 1007 rick 0.0 0.1 grep --color=auto -E subprocess</pre>

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command	Output
<code>ps --forest -C mainprocess -C subprocess1 -C subprocess2</code>	<pre>rick@secr2043:~\$ ps --forest -C mainprocess -C subprocess1 -C subprocess2 PID TTY TIME CMD 906 pts/0 00:00:00 mainprocess 907 pts/0 00:00:00 _ mainprocess 909 pts/0 00:00:00 _ subprocess1 913 pts/0 00:00:00 _ subprocess1 908 pts/0 00:00:00 _ mainprocess 910 pts/0 00:00:00 _ subprocess2 911 pts/0 00:00:00 _ subprocess2 912 pts/0 00:00:00 _ subprocess2</pre>

Question 7

Use `ptree` command with option that show the number of threads to each process.

Command	Output
pstree -c -s 906	rich@secr2003:~\$ pstree -c -s 906 systemd--sshd---sshd---sshd---bash---mainprocess+-mainprocess+-subprocess1 -mainprocess+-subprocess1 -subprocess2 -subprocess2

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command	Output
<code>sudo renice -5 909</code>	<pre>rick@secr2043:~\$ ps -o pid,nice,comm -C subprocess1 PID NI COMMAND 909 0 subprocess1 913 0 subprocess1 rick@secr2043:~\$ sudo renice -5 909 [sudo] password for rick: 909 (process ID) old priority 0, new priority -5</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command	Output
<code>killall -15 mainprocess</code>	<pre>rick@secr2043:~\$ killall -15 mainprocess Main process (ID: 907) received signal: 15. Terminating... Main process (ID: 906) received signal: 15. Terminating... Main process (ID: 908) received signal: 15. Terminating... [1]+ Done ./mainprocess</pre>

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code: nano example.c

`gcc example.c -o example`

`gcc./example`

`example.c`

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
void run_child(int child_num) {
```

```
    printf("Child %d (PID: %d) started\n", child_num, getpid());
```

```
    sleep(child_num); // sleep for 1 second for child 1, 2 seconds for child 2
```

```
    printf("Child %d (PID: %d) finished sleeping\n", child_num, getpid());
```

```
    exit(child_num); // exit with status = child_num
```

```
}
```

```
int main() {
```

```
    printf("Parent process (PID: %d) starting\n", getpid());
```

```
    for (int i = 1; i <= 2; i++) {
```

```
        pid_t pid = fork();
```

```
        if (pid == 0) {
```

```
            run_child(i);
```

```
    } else if (pid < 0) {  
        perror("Fork failed");  
        exit(1);  
    }  
}
```

Output:

```
rick@secr2043:~$ nano example.c  
rick@secr2043:~$ gcc example.c -o example  
rick@secr2043:~$ ./example  
Parent process (PID: 1081) starting  
Child 1 (PID: 1082) started  
Child 2 (PID: 1083) started  
Child 1 (PID: 1082) finished sleeping  
Parent: Child with PID 1082 exited with status 1  
Child 2 (PID: 1083) finished sleeping  
Parent: Child with PID 1083 exited with status 2  
Parent process (PID: 1081) done
```

